Niagara Particle System Assignment - Cansu Öner

I made a firefighting prototype in Unreal Engine 5.7 with GPU sprite particles for fire and CPU mesh particles for water. The player aims with the camera and sprays water with the left mouse button. Water gradually extinguishes the fire it touches, with steam appearing at the contact point. I also added a separate spray emitter to make the water outlet look better.

Setup and design choices

I split the fire into smaller regions so that spraying one area does not extinguish everything at once. Each region has its own strength, which controls User.FireSpawnRate. GPU simulation lets me use a large number of sprites to create dense fire. The base emmitter is a fountain emitter. I changed parameters like the gravity, velocity and color scale to make it look like fire.

The main water emitter uses mesh particles with a one-second lifetime, initial velocity, gravity, drag and collision. The droplets bounce against surfaces and export their positions to Blueprint, where I check whether they reach a fire region. The additional spray uses sprites without exporting particle data. The emitters are again configured fouintain emitters.

Performance tests

I checked performance using stat unit, stat Niagara, GPU profiling and the Niagara Debugger. Water was the biggest limitation:

- Around 2,000 water particles worked reasonably well.
- Around 3,000 water particles caused low FPS, even with only 80,000 fire particles.
- Around 5,000 water particles dropped performance to approximately 12 FPS.

Before optimization, some tests with around 120,000 fire particles also ran poorly. After optimization, I could run approximately 237,000 fire particles with 2,000 water particles at around 40 FPS. These tests had different conditions, so the numbers are approximate.

When spraying water, Game-thread time increased significantly while GPU time stayed around 12–15 ms. Disabling particle export brought performance back to around 50 FPS. Keeping export enabled but skipping the Blueprint search gave a similar result, suggesting that the interaction checks were the main bottleneck.

Optimization and results

For fire, I disabled gravity, drag and curl noise, and used fixed initial speed and lifetime. This kept the upward movement and fading while simplifying the simulation.

For water, I cached the exported particle array and ran the interaction checks less often. The search stops once it finds contact for a fire region and skips extinguished regions. After these changes, I observed roughly 30–40 FPS instead of 10–20 FPS in the tested configuration.

I did not consistently reach my 60 FPS target. The main finding was that a few thousand water particles with collision and Blueprint interaction could cost more than hundreds of thousands of simple GPU fire sprites.